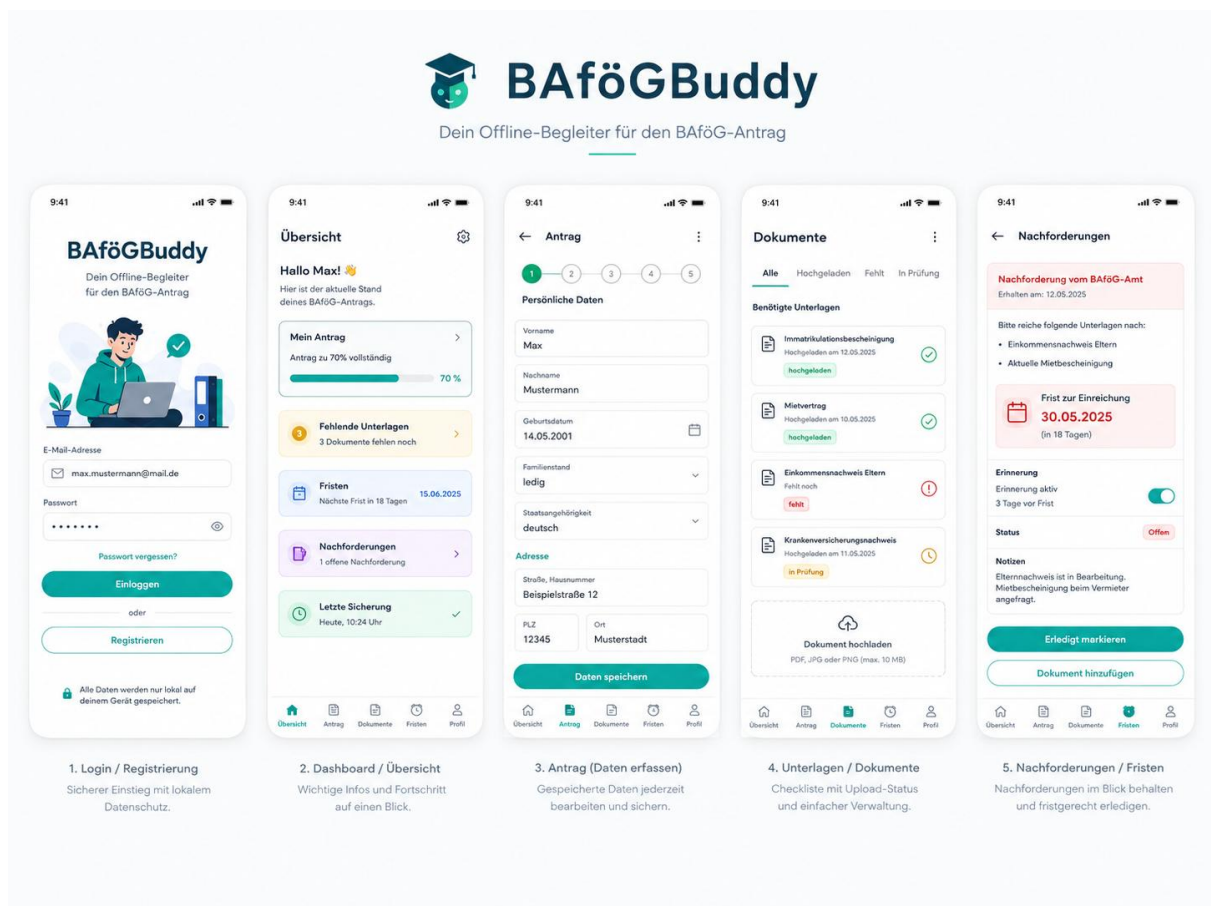


BAföGBuddy

Technical White Paper - Meilenstein 3

Finales Produkt + Qualitätssicherung

Projekt	BAföGBuddy - Offline-Assistent für BAföG-Anträge
Student	Jad El Jakhlab
Matrikelnummer	599861
Stand	11.06.2026
Abgabe	Meilenstein 3 (finales Produkt + Qualitätssicherung)



Hinweis: BAföGBuddy ersetzt nicht das offizielle BAföG-Portal. Die App unterstützt die Vorbereitung, Verwaltung und Nachverfolgung von Antragsdaten, Unterlagen, Nachforderungen und Fristen.

1. Projektidee

BAföGBuddy ist eine offline-fähige Android-App, die Studierende bei der Vorbereitung und Verwaltung von BAföG-Anträgen unterstützt. Die App hilft dabei, persönliche Daten, benötigte Unterlagen, Nachforderungen und Fristen übersichtlich zu verwalten. Ziel ist es, fehlende Dokumente frühzeitig zu erkennen und Folgeanträge einfacher vorzubereiten.

Die offizielle Einreichung erfolgt weiterhin über BAföG Digital oder das zuständige Amt. BaföGBuddy ist also kein Ersatz für das offizielle Portal, sondern ein organisatorischer Assistent vor, während und nach der Antragstellung.

2. Ziel von Meilenstein 3

In Meilenstein 3 wurde die Anwendung nach dem API-first-Ansatz umgesetzt. Im Mittelpunkt stehen Komponenten, Schnittstellen und Testbarkeit.

- Die App wird in klar getrennte Komponenten zerlegt.
- Jede Komponente bekommt eine eindeutige Aufgabe.
- Jede zentrale Komponente bietet eine definierte Schnittstelle/API.
- Die Komponenten wurden so umgesetzt, dass sie automatisiert testbar sind.

3. Architekturidee: MVC

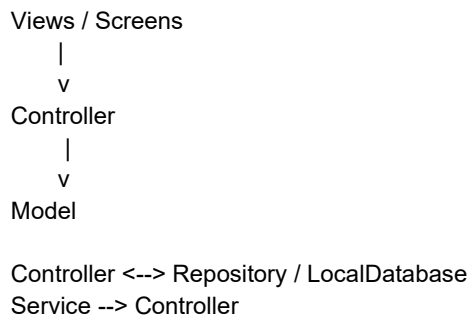
BAföGBuddy wird grob nach MVC strukturiert. Diese Trennung macht die App wartbarer und testbarer.

Bereich	Bedeutung in BaföGBuddy	Beispiele
Model	Daten der App	User, BafogApplication, DocumentItem, Deadline, FollowUpRequest
View	Benutzeroberfläche	LoginScreen, DashboardScreen, DocumentsScreen, DeadlinesScreen
Controller	Fachlogik der App	RequirementChecker, DocumentController, DeadlineController, DashboardController, FollowUpController

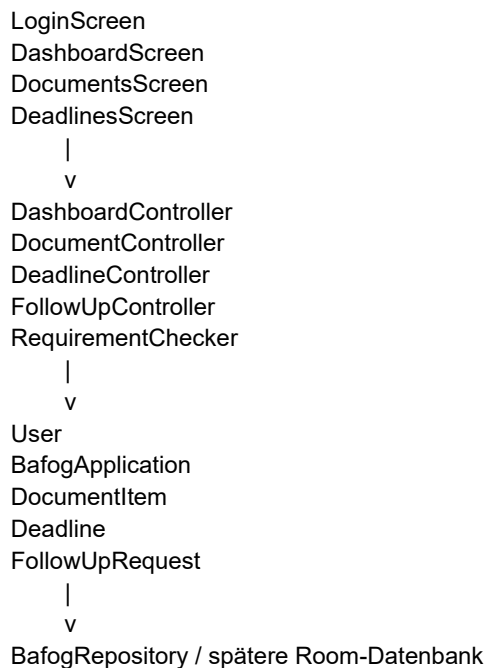
4. Komponentenübersicht

Komponente	Paket	Aufgabe
User	model	Speichert einfache Nutzerdaten
BafogApplication	model	Beschreibt einen BAföG-Antrag
DocumentItem	model	Beschreibt eine benötigte Unterlage
Deadline	model	Beschreibt eine Frist
FollowUpRequest	model	Beschreibt eine Nachforderung vom BAföG-Amt
RequirementChecker	controller	Prüft fehlende Pflichtunterlagen und berechnet den Fortschritt
DocumentController	controller	Ändert den Status von Unterlagen
DeadlineController	controller	Berechnet offene und kritische Fristen
FollowUpController	controller	Verwaltet Nachforderungen
DashboardController	controller	Erstellt die Übersicht für das Dashboard
BafogRepository	data	Definiert die Schnittstelle zur späteren lokalen Datenbank
ReminderService	service	Prüft, ob eine Erinnerung ausgelöst werden soll
NotificationService	service	Erstellt Texte für spätere Benachrichtigungen

5. Komponentendiagramm



Ausführlicher:



6. API-first: geplante Schnittstellen

RequirementChecker

Aufgabe: Prüft, ob ein BAföG-Antrag vollständig ist.

```
getMissingRequiredDocuments(documents)
calculateCompletionPercentage(documents)
isApplicationComplete(documents)
```

Testidee:

- Fehlende Pflichtunterlagen werden korrekt erkannt.
- Optionale fehlende Dokumente zählen nicht als Problem.
- Der Fortschritt wird korrekt berechnet.

DocumentController

Aufgabe: Verwaltet den Status von Unterlagen.

```
markAsAvailable(document)
markAsUploaded(document)
markAsMissing(document)
markAsInReview(document)
```

Testidee:

- Ein Dokument kann von fehlt zu hochgeladen wechseln.
- Der Status wird korrekt geändert.

DeadlineController

Aufgabe: Verwaltet Fristen.

```
getOpenDeadlines(deadlines)
calculateDaysLeft(deadline, today)
isDeadlineCritical(deadline, today)
```

Testidee:

- Erledigte Fristen werden nicht als offen angezeigt.
- Die verbleibenden Tage werden korrekt berechnet.
- Eine Frist wird als kritisch erkannt, wenn nur noch wenige Tage bleiben.

FollowUpController

Aufgabe: Verwaltet Nachforderungen vom BAföG-Amt.

```
getOpenFollowUps(followUps)
markAsDone(followUp)
markAsInProgress(followUp)
```

Testidee:

- Offene Nachforderungen werden korrekt angezeigt.
- Nachforderungen können als erledigt markiert werden

DashboardController

Aufgabe: Erstellt die Übersicht für die Startseite

```
buildSummary(documents, deadlines, followUps)
```

Testidee:

- Fortschritt, fehlende Unterlagen, offene Fristen und offene Nachforderungen werden korrekt zusammengefasst.

7. Bereits umgesetzte Tests

Testklasse	Geteste Komponente	Zweck
RequirementCheckerTest	RequirementChecker	Fehlende Unterlagen und Fortschritt prüfen
DeadlineControllerTest	DeadlineController	Fristen und kritische Fristen prüfen
DashboardControllerTest	DashboardController	Dashboard-Zusammenfassung prüfen
DocumentControllerTest	DocumentController	Dokumentstatus ändern und prüfen
FollowUpControllerTest	FollowUpController	Nachforderungen filtern und Status ändern

Alle Unit-Tests wurden erfolgreich ausgeführt. Zusätzlich wurde der komplette Gradle-Build erfolgreich ausgeführt.

8. Erklärung der aktuellen Projektstruktur

model = Daten der App
controller = Fachlogik der App
data = spätere Speicherung / Datenbank
service = Erinnerungen und Benachrichtigungen
view = geplante Oberflächen

Diese Struktur hilft dabei, die App testbar und wartbar zu machen. Die zentrale Fachlogik liegt nicht direkt in der Android-Oberfläche, sondern in separaten Controller-Komponenten. Dadurch können Unit-Tests unabhängig von der GUI ausgeführt werden.

9. Technischer Schwerpunkt

Der technische Schwerpunkt von BAföGBuddy liegt auf lokaler Datenhaltung, offline nutzbarer Dokumenten- und Fristenverwaltung, testbarer Fachlogik und Vorbereitung einer späteren Datenbankanbindung.

- Kotlin als Programmiersprache
- Android Views/XML als erste GUI-Grundlage
- JUnit für Unit-Tests
- Repository Pattern als Vorbereitung für Room Database
- spätere Reminder/Notifications für Fristen

10. Umgesetzter Prototyp

Für Meilenstein 3 wurde ein lauffähiger Android-Prototyp umgesetzt.

Die Anwendung startet erfolgreich im Android Emulator und zeigt eine erste BAföGBuddy-Oberfläche.

Zusätzlich wurden Controller, Modelle, Repository-Schnittstellen sowie automatisierte Unit-Tests implementiert.

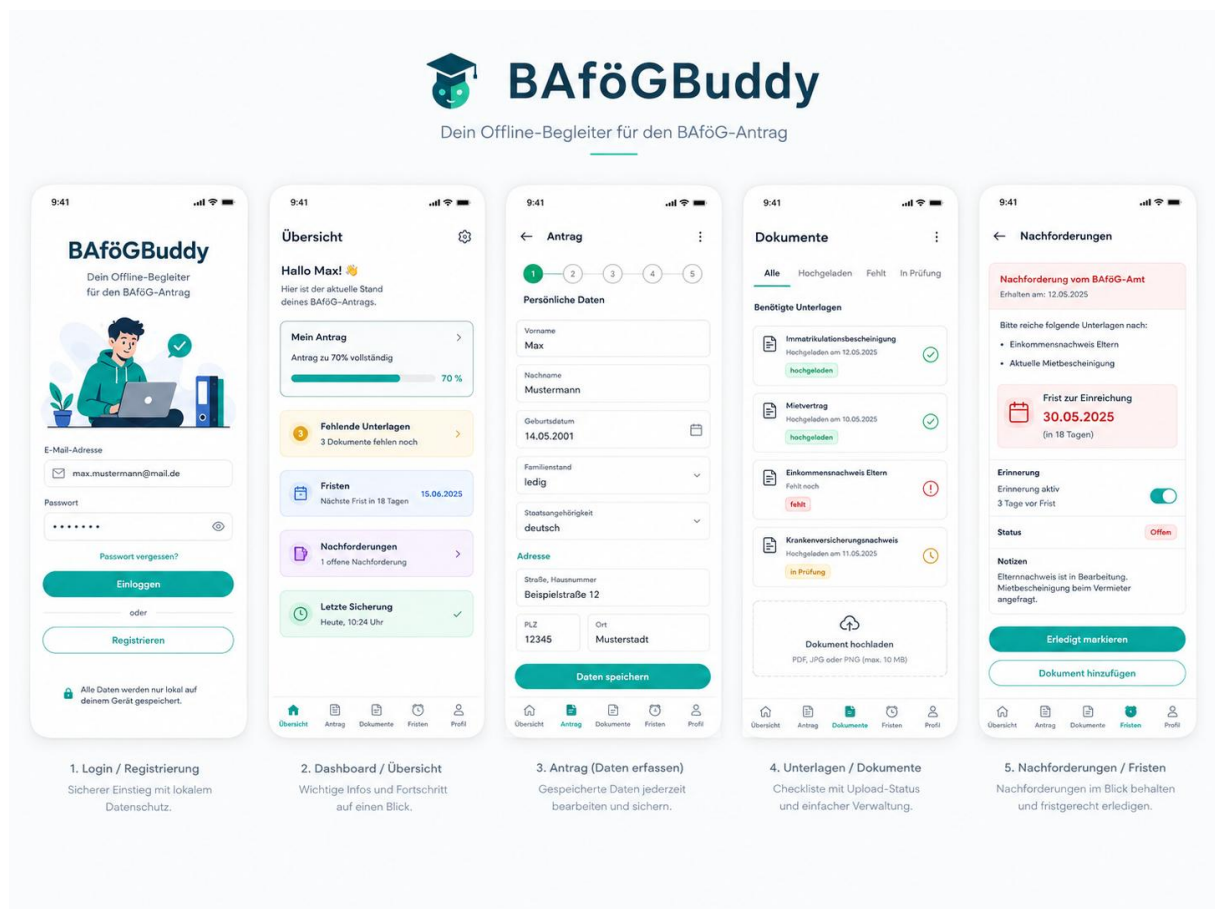
Der aktuelle Stand dient als funktionaler Prototyp und Grundlage für zukünftige Erweiterungen.

GitHub Repository:

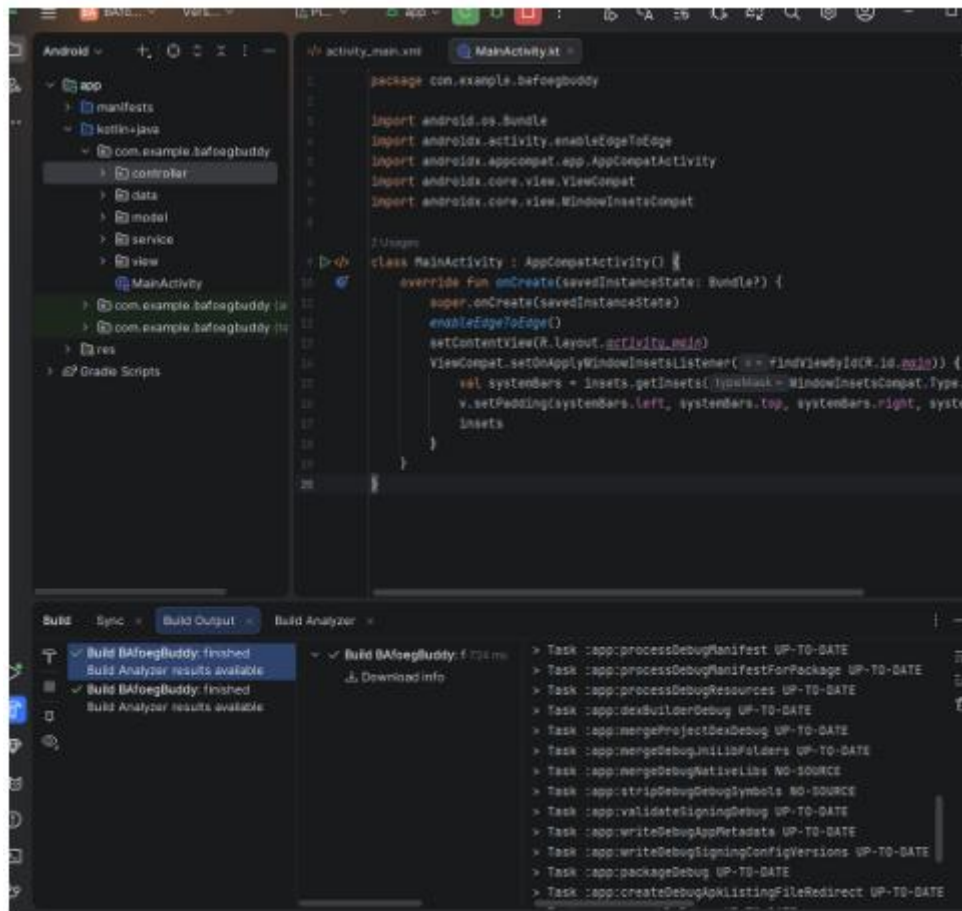
<https://github.com/eljakj/BAfoegBuddy>

11. Nachweise

GUI-Mockup der geplanten App:



Nachweis: Projektstruktur und Build-Ausgabe in Android Studio



Nachweis: Gradle Build erfolgreich

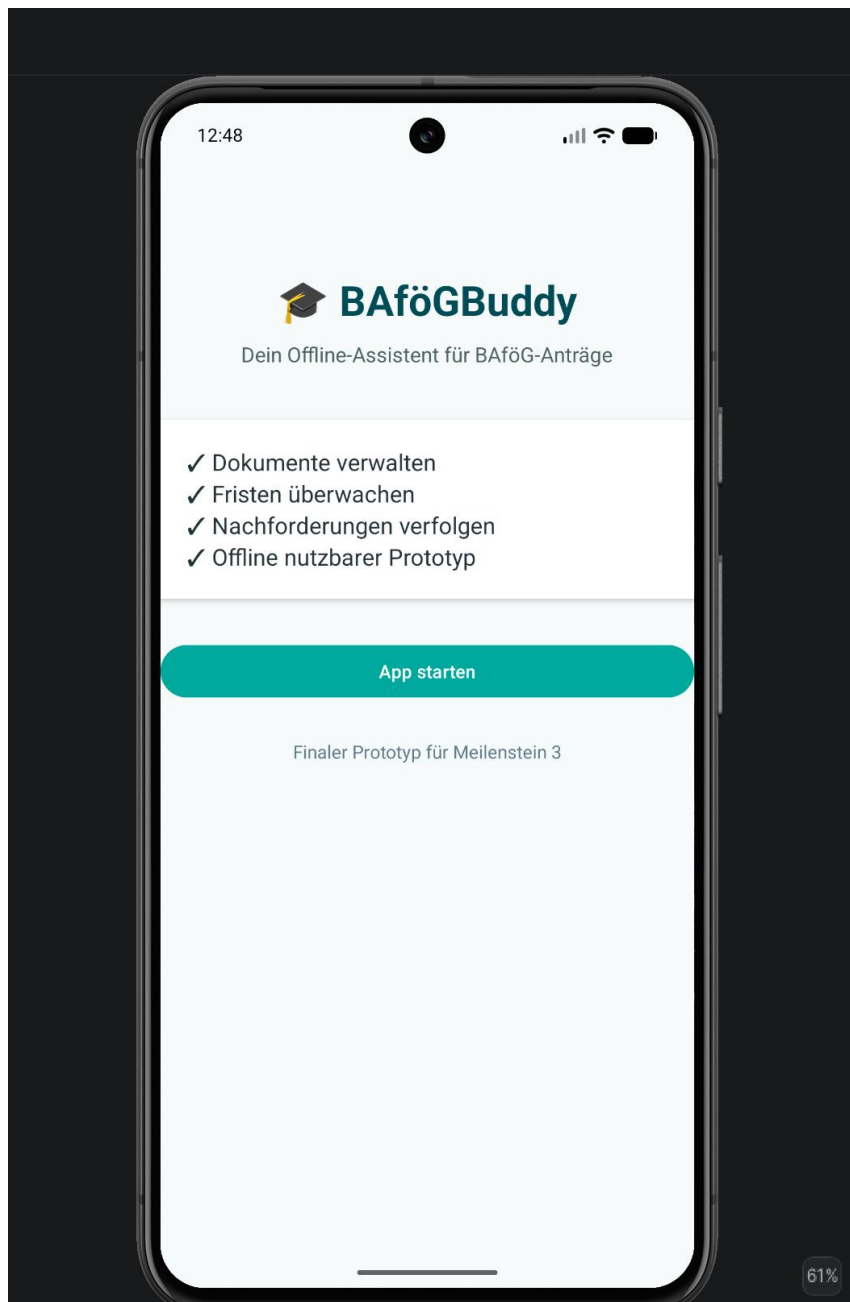
```
PS C:\Users\admin\Documents\AndroidStudioProjects\BAfoegBuddy> .\gradlew build

> Task :app:lintReportDebug
Wrote HTML report to file:///C:/Users/admin/Documents/AndroidStudioProjects/BAfoegBuddy/app/build/reports/lint-results-debug.html

BUILD SUCCESSFUL in 37s
89 actionable tasks: 52 executed, 37 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.1/userguide/configuration\_cache\_enabling.html

PS C:\Users\admin\Documents\AndroidStudioProjects\BAfoegBuddy> 
```

Nachweis: BAföGBuddy erfolgreich im Android Emulator gestartet



12. Fazit

BAföGBuddy wurde erfolgreich als Android-Prototyp umgesetzt.

Die Anwendung besitzt eine MVC-Struktur, API-first Planung, testbare Komponenten und automatisierte Unit Tests.

Der aktuelle Stand erfüllt die Anforderungen von Meilenstein 3 (Finales Produkt + Qualitätssicherung).